

**VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**



**WEB APPLICATION DEVELOPMENT
IT093IU
LAB 4 (Homework)**

Member List:

- Huỳnh Ngọc Bảo Long – ITITWE23023

Instructor: MSc. Nguyễn Trung Nghĩa

Exercise 5

Demo/Test cases:

Student Management System

Add New Student

Search by name or code...

Search

Clear

ID	Student Code	Full Name	Email	Major	Created At	Actions
7	SV007	Huynh Ngoc Bao Long	Long@gmail.com	CSE	2025-11-08 15:44:05.0	<a>Edit <a>Delete
6	SV006	Huynh Ngoc Bao Long	Long@gmail.com	CSE	2025-11-08 15:42:56.0	<a>Edit <a>Delete
5	SV005	David Wilson	david.w@email.com	Computer Science	2025-11-08 13:53:15.0	<a>Edit <a>Delete
4	SV004	Sarah Davis	sarah.d@email.com	Data Science	2025-11-08 13:53:15.0	<a>Edit <a>Delete
3	SV003	Michael Brown	michael.b@email.com	Software Engineering	2025-11-08 13:53:15.0	<a>Edit <a>Delete
2	SV002	Emily Johnson	emily.j@email.com	Information Technology	2025-11-08 13:53:15.0	<a>Edit <a>Delete
1	SV001	John Doe	john.smith@email.com	Computer Science	2025-11-08 13:53:15.0	<a>Edit <a>Delete

Search by name or code...

Search

Clear

ID

Student Code

Full Name

Search term “John” → Shows all students with “John” in name:

John

Search

Clear

ID	Student Code	Full Name	Email	Major	Created At	Actions
2	SV002	Emily Johnson	emily.j@email.com	Information Technology	2025-11-08 13:53:15.0	<a>Edit <a>Delete
1	SV001	John Doe	john.smith@email.com	Computer Science	2025-11-08 13:53:15.0	<a>Edit <a>Delete

localhost:8080/StudentManagement/list_students.jsp?keyword=John

Search term “SV001” → Shows student with code SV001:

SV001

Search

Clear

ID	Student Code	Full Name	Email	Major	Created At	Actions
1	SV001	John Doe	john.smith@email.com	Computer Science	2025-11-08 13:53:15.0	<a>Edit <a>Delete

localhost:8080/StudentManagement/list_students.jsp?keyword=SV001

Search term “science” → Shows students in Computer Science/Data Science:

Science

Search

Clear

ID	Student Code	Full Name	Email	Major	Created At	Actions
5	SV005	David Wilson	david.w@email.com	Computer Science	2025-11-08 13:53:15.0	Edit Delete
4	SV004	Sarah Davis	sarah.d@email.com	Data Science	2025-11-08 13:53:15.0	Edit Delete
1	SV001	John Doe	john.smith@email.com	Computer Science	2025-11-08 13:53:15.0	Edit Delete

localhost:8080/StudentManagement/list_students.jsp?keyword=Science

Search term “” (empty) → Shows all students:

Search by name or code...

Search

Clear

ID	Student Code	Full Name	Email	Major	Created At	Actions
7	SV007	Huynh Ngoc Bao Long	Long@gmail.com	CSE	2025-11-08 15:44:05.0	Edit Delete
6	SV006	Huynh Ngoc Bao Long	Long@gmail.com	CSE	2025-11-08 15:42:56.0	Edit Delete
5	SV005	David Wilson	david.w@email.com	Computer Science	2025-11-08 13:53:15.0	Edit Delete
4	SV004	Sarah Davis	sarah.d@email.com	Data Science	2025-11-08 13:53:15.0	Edit Delete
3	SV003	Michael Brown	michael.b@email.com	Software Engineering	2025-11-08 13:53:15.0	Edit Delete
2	SV002	Emily Johnson	emily.j@email.com	Information Technology	2025-11-08 13:53:15.0	Edit Delete
1	SV001	John Doe	john.smith@email.com	Computer Science	2025-11-08 13:53:15.0	Edit Delete

localhost:8080/StudentManagement/list_students.jsp?keyword=

Explanation:

- User interaction
 - The search box is part of the same list_students.jsp page.
 - When the user types a search term (e.g., science) and presses Enter or clicks Search, the form submits the keyword using the GET method.
 - The browser appends the search term to the URL as a query parameter: list_students.jsp?keyword=science
- Receiving the keyword
 - The JSP retrieves the search term from the request: `String keyword = request.getParameter("keyword");`
 - If the user has entered something (non-empty keyword), the page performs a filtered SQL query; otherwise, it displays all students.

- Database query preparation
 - The JSP constructs a parameterized SQL query using PreparedStatement to prevent SQL injection:
`sql = "SELECT * FROM students WHERE full_name LIKE ? OR student_code LIKE ? OR major LIKE ? ORDER BY id DESC";`
`pstmt = conn.prepareStatement(sql);`
 - It binds the search value (with wildcards) to all three parameters:
`pstmt.setString(1, "%" + keyword + "%");`
`pstmt.setString(2, "%" + keyword + "%");`
`pstmt.setString(3, "%" + keyword + "%");`
- Database execution and filtering
 - MySQL executes the query and returns only the rows that contain the keyword in any of the columns:
 - full_name
 - student_code
 - major
 - For example: searching "science" now matches "Computer Science" or "Data Science" majors.
- Displaying filtered results
 - The JSP iterates through the ResultSet and displays only the matching students in the table.
 - If no matches are found, the page can show a “No results found” message (optional).
- GET method behavior
 - Since the form uses the GET method, the search keyword remains visible in the browser’s address bar.
 - This allows users to bookmark or share the search URL, e.g.:
`list_students.jsp?keyword=science`

Exercise 6

Test cases:

- Email validation:
 Input: john@email.com → Valid, accepts

Add New Student

Student Code *

TEST1234

Full Name *

Test Name

Email

john@email.com

Major

Computer Science



Save Student

Cancel



Student Management System

Student added successfully



Add New Student

Input: john.doe@company.co.uk → Valid, accepts

Edit Student Information

Student Code

Cannot be changed

Full Name *

Email

Major

 Update

Cancel

Student updated successfully

[+ Add New Student](#)[Search](#)[Clear](#)

ID	Student Code	Full Name	Email	Major	Created At	Actions
12	SV1111	Test Name	john.doe@company.co.uk	Computer Science	2025-11-13 20:39:57.0	Edit Delete

Input: john@email → Invalid, rejects

Add New Student

Invalid email format

Student Code *

SV1111

Full Name *

Test Name

Email

john@email

Major

Computer Science



Save Student

Cancel

Input: johnemail.com → Invalid, rejects

+ Add New Student

Invalid email format

Student Code *

SV1111

Full Name *

Test Name

Email

johnemail.com

Major

Computer Science



Save Student

Cancel



Please include an '@' in the email address. 'johnemail.com' is missing an '@'.

Input: (empty) → Valid, accepts (optional field)

ID	Student Code	Full Name	Email	Major	Created At	Actions
14	SV1111	Test Name		Computer Science	2025-11-13 20:43:16.0	Edit Delete

- Student code validation:
Valid:

+ Add New Student

Student Code *

Full Name *

Email

Major

 Save Student

Cancel

localhost:8080/StudentManagement/list_students.jsp?message=Student%20added%20successfully

Student Management System

Student added successfully

[+ Add New Student](#)

Search by name or code... [Search](#) [Clear](#)

ID	Student Code	Full Name	Email	Major	Created At	Actions
11	IT123	Test Name	testmail@gmail.com	Computer Science	2025-11-13 20:37:55.0	Edit Delete
7	SV007	Huynh Ngoc Bao Long	Long@gmail.com	CSE	2025-11-08 15:44:05.0	Edit Delete
6	SV006	Huynh Ngoc Bao Long	Long@gmail.com	CSE	2025-11-08 15:42:56.0	Edit Delete
5	SV005	David Wilson	david.w@email.com	Computer Science	2025-11-08 13:53:15.0	Edit Delete
4	SV004	Sarah Davis	sarah.d@email.com	Data Science	2025-11-08 13:53:15.0	Edit Delete
3	SV003	Michael Brown	michael.b@email.com	Software Engineering	2025-11-08 13:53:15.0	Edit Delete
2	SV002	Emily Johnson	emily.j@email.com	Information Technology	2025-11-08 13:53:15.0	Edit Delete
1	SV001	John Doe	john.smith@email.com	Computer Science	2025-11-08 13:53:15.0	Edit Delete

Invalid:

Student Code *

sv001

Add New Student

Student Code *

sv001

Full Name *

Test Name



Please match the requested format.

Format: 2 uppercase letters + 3+ digits

Email

testmail@gmail.com

Major

Computer Science



Save Student

Cancel

Add New Student

Student Code *

S001

Full Name *

Test Name



Please match the requested format.

Format: 2 uppercase letters + 3+ digits

Email

testmail@gmail.com

Major

Computer Science



Save Student

Cancel

+ Add New Student

Student Management System

[Add New Student](#)

Search by name, code, or major... [Search](#) [Clear](#)

#	Student Code	Full Name	Email	Major	Created At	Actions
1	SV013	Test Name	test@email.com	Computer Science	2025-11-13 20:59:12.0	Edit Delete
2	SV012	Test Name	test@email.com	Computer Science	2025-11-13 20:55:21.0	Edit Delete
3	SV011	Test Name	test@email.com	Computer Science	2025-11-13 20:55:13.0	Edit Delete
4	SV010	Test Name	test@email.com	Computer Science	2025-11-13 20:55:08.0	Edit Delete
5	SV009	Test Name	test@email.com	Computer Science	2025-11-13 20:54:57.0	Edit Delete
6	SV008	Test Name	test@email.com	Computer Science	2025-11-13 20:54:52.0	Edit Delete
7	SV007	Huynh Ngoc Bao Long	Long@gmail.com	CSE	2025-11-08 15:44:05.0	Edit Delete
8	SV006	Huynh Ngoc Bao Long	Long@gmail.com	CSE	2025-11-08 15:42:56.0	Edit Delete
9	SV005	David Wilson	david.w@email.com	Computer Science	2025-11-08 13:53:15.0	Edit Delete
10	SV004	Sarah Davis	sarah.d@email.com	Data Science	2025-11-08 13:53:15.0	Edit Delete

1 2 [Next](#)

Explanation:

- User interaction
 - The search box and student table are on the same list_students.jsp page.
 - The user types a search term (e.g., "science") and clicks Search or presses Enter.
 - The form submits the keyword using the GET method, appending it to the URL: list_students.jsp?keyword=science&page=1
 - When the user clicks a pagination link (Previous, Next, or page number), the page reloads with the updated page query parameter, e.g., list_students.jsp?keyword=science&page=2.
- Receiving input in JSP
 - The JSP retrieves the search term and page number from the request:


```
String keyword = request.getParameter("keyword");
String pageParam = request.getParameter("page");
int currentPage = (pageParam != null) ? Integer.parseInt(pageParam) : 1;
```
 - If the keyword is empty or null, all students are displayed; otherwise, a filtered query is executed.
- Pagination calculation
 - The JSP calculates the offset for SQL LIMIT using the current page and records per page:

```
int recordsPerPage = 10;
```

```
int offset = (currentPage - 1) * recordsPerPage;
```

- The total number of pages is computed from the total student records:

```
int totalPages = (int) Math.ceil((double) totalRecords / recordsPerPage);
```

- Database query preparation

- The JSP prepares a parameterized SQL query using PreparedStatement to prevent SQL injection:

```
SELECT * FROM students
```

```
WHERE full_name LIKE ? OR student_code LIKE ? OR major LIKE ?
```

```
ORDER BY id DESC
```

```
LIMIT ? OFFSET ?
```

- The search keyword is bound to all three parameters with wildcards (%) for partial matching:

```
pstmt.setString(1, "%" + keyword + "%");
```

```
pstmt.setString(2, "%" + keyword + "%");
```

```
pstmt.setString(3, "%" + keyword + "%");
```

```
pstmt.setInt(4, recordsPerPage);
```

```
pstmt.setInt(5, offset);
```

- Database execution and filtering

- MySQL executes the query and returns only the rows matching the keyword, considering full_name, student_code, or major.
- Pagination ensures that only 10 records per page are retrieved.

- Displaying results in JSP

- The JSP iterates through the ResultSet and displays the students in a responsive table.
- Row numbers are calculated dynamically using the offset so that numbering is continuous across pages:

```
int rowNumber = offset + 1;
```

- If no results are found, the table remains empty (optionally a "No results found" message can be displayed).
- Enhanced UX features
 - Success and error messages are styled with icons (✓ / ✗), colored backgrounds, and automatically hide after 3 seconds using JavaScript.
 - The search button disables and shows "Processing..." while the query executes, preventing double submissions.
 - The table is responsive on smaller screens, scrollable horizontally, and font/padding adjusts for mobile devices.
- GET method benefits
 - The search term and page number remain visible in the browser's URL.
 - Users can bookmark or share specific search results and pages:
list_students.jsp?keyword=science&page=2